```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
import math
```

```
uploaded = files.upload()

video_path = list(uploaded.keys())[0]

print("Video uploaded successfully:", video_path)
```

```
Choose Files vlipsy-wolv…iF1ESc.mp4
vlipsy-wolverine-smashes-alarm-clock-YZiF1ESc.mp4(video/mp4) - 283264 bytes, last modified: 9/24/2026 - 100% done
Saving vlipsy-wolverine-smashes-alarm-clock-YZiF1ESc.mp4 to vlipsy-wolverine-smashes-alarm-clock-YZiF1ESc.mp4
Video uploaded successfully: vlipsy-wolverine-smashes-alarm-clock-YZiF1ESc.mp4
```

```
cap = cv2.VideoCapture(video_path)

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

duration = total_frames / fps if fps > 0 else 0

print("Video Width:", width)
print("Video Height:", height)
print("FPS:", fps)
print("Total Frames:", total_frames)
print("Duration: {:.2f} seconds".format(duration))

cap.release()
```

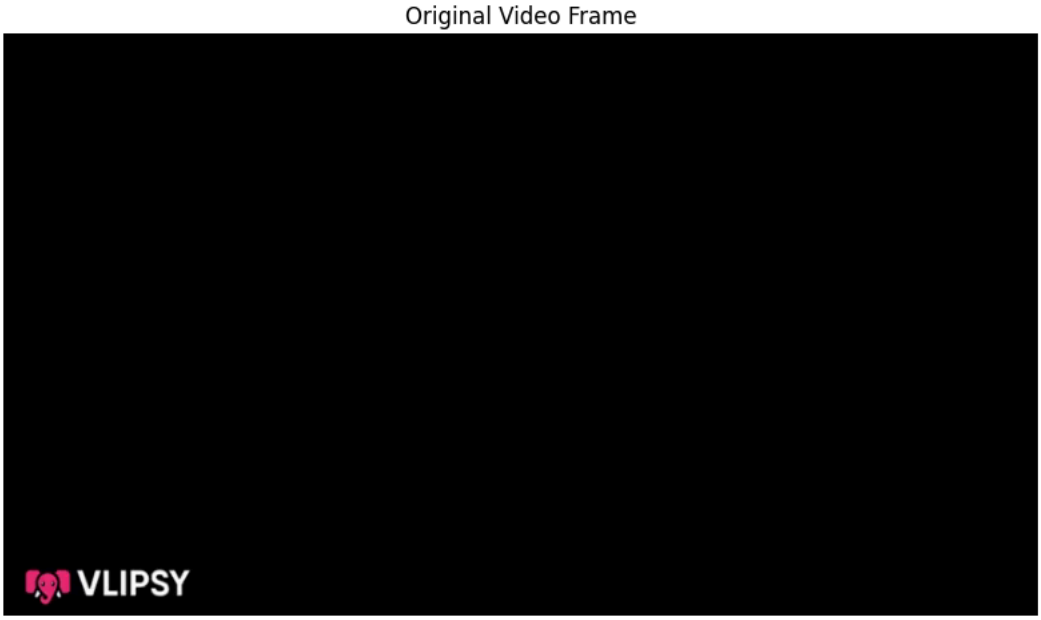
```
Video Width: 854
Video Height: 480
FPS: 30.0
Total Frames: 150
Duration: 5.00 seconds
```

```
cap = cv2.VideoCapture(video_path)

ret, frame = cap.read()
cap.release()

if ret:
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(10, 6))
    plt.imshow(frame_rgb)
    plt.title("Original Video Frame")
    plt.axis("off")
    plt.show()
else:
    print("Unable to read video.")
```




```
cap = cv2.VideoCapture(video_path)

ret, frame = cap.read()
cap.release()

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

corners = cv2.goodFeaturesToTrack(
    gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

corner_frame = frame.copy()

if corners is not None:
    corners = np.int32(corners)

    for corner in corners:
        x, y = corner.ravel()

        cv2.circle(
            corner_frame,
            (x, y),
            5,
            (0, 255, 0),
            -1
        )

corner_rgb = cv2.cvtColor(corner_frame, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 6))
plt.imshow(corner_rgb)
plt.title("Shi-Tomasi Corner Detection")
plt.axis("off")
plt.show()

print("Number of detected feature points:",
      0 if corners is None else len(corners))
```




```
Number of detected feature points: 16
```

```
# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

# Shi-Tomasi parameters
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

cap = cv2.VideoCapture(video_path)

ret, old_frame = cap.read()

if not ret:
    print("Unable to read video.")
else:

    old_gray = cv2.cvtColor(
        old_frame,
        cv2.COLOR_BGR2GRAY
    )

    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )

    # Mask for trajectories
    mask = np.zeros_like(old_frame)

    trajectory_data = []
    frame_number = 0

    while True:

        ret, frame = cap.read()

        if not ret:
            break

        frame_gray = cv2.cvtColor(
            frame,
            cv2.COLOR_BGR2GRAY
```

```
        cv2.COLOR_BGR2GRAY
    )

    if p0 is None:
        break

    # Calculate optical flow
    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
        **lk_params
    )

    if p1 is None:
        break

    good_new = p1[status == 1]
    good_old = p0[status == 1]

    displacements = []

    for new, old in zip(good_new, good_old):

        x_new, y_new = new.ravel()
        x_old, y_old = old.ravel()

        # Calculate displacement
        displacement = math.sqrt(
            (x_new - x_old) ** 2 +
            (y_new - y_old) ** 2
        )

        displacements.append(displacement)

        # Draw trajectory
        mask = cv2.line(
            mask,
            (int(x_new), int(y_new)),
            (int(x_old), int(y_old)),
            (0, 255, 0),
            2
        )

        # Draw current point
        frame = cv2.circle(
            frame,
            (int(x_new), int(y_new)),
            5,
            (0, 0, 255),
            -1
        )

    if len(displacements) > 0:

        avg_displacement = np.mean(
            displacements
        )

        trajectory_data.append(
            avg_displacement
        )

        # Average direction
        dx = np.mean(
            good_new[:, 0] -
            good_old[:, 0]
        )

        dy = np.mean(
            good_new[:, 1] -
            good_old[:, 1]
        )

        direction = math.degrees(
            math.atan2(dy, dx)
        )

        cv2.putText(
            frame,
            f"Displacement: {avg_displacement:.2f} px",
            (10, 30),
```

```
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (0, 255, 0),
                2
            )

            cv2.putText(
                frame,
                f"Direction: {direction:.2f} degrees",
                (10, 60),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (0, 255, 0),
                2
            )

        # Combine frame and trajectory
        output = cv2.add(
            frame,
            mask
        )

        # Display selected frames
        if frame_number % 10 == 0:

            output_rgb = cv2.cvtColor(
                output,
                cv2.COLOR_BGR2RGB
            )

            plt.figure(figsize=(10, 6))
            plt.imshow(output_rgb)
            plt.title(
                f"Lucas-Kanade Optical Flow - Frame {frame_number}"
            )
            plt.axis("off")
            plt.show()

        # Update
        old_gray = frame_gray.copy()

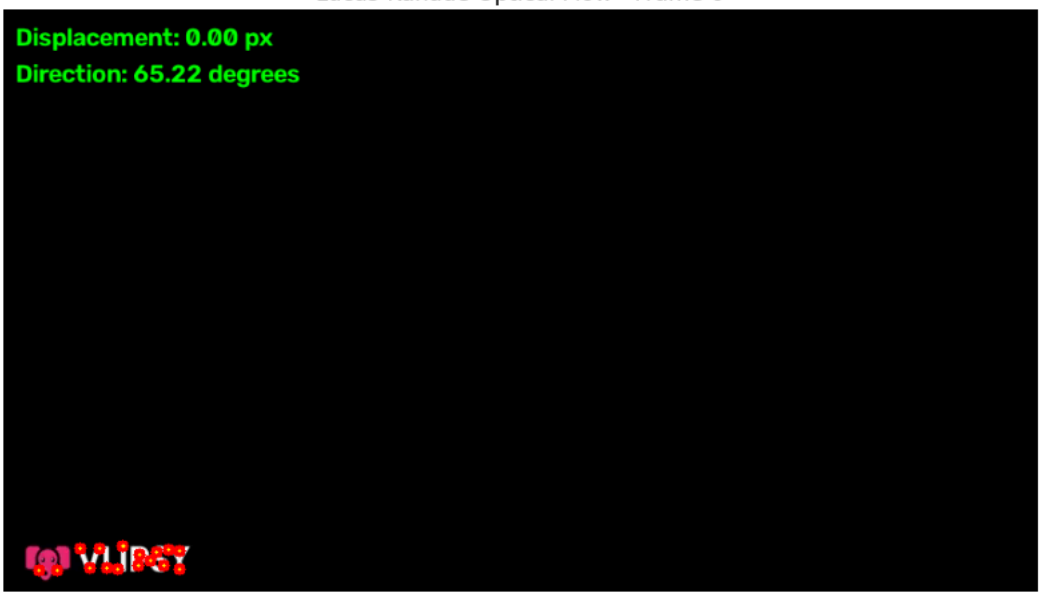
        p0 = good_new.reshape(
            -1,
            1,
            2
        )

        frame_number += 1

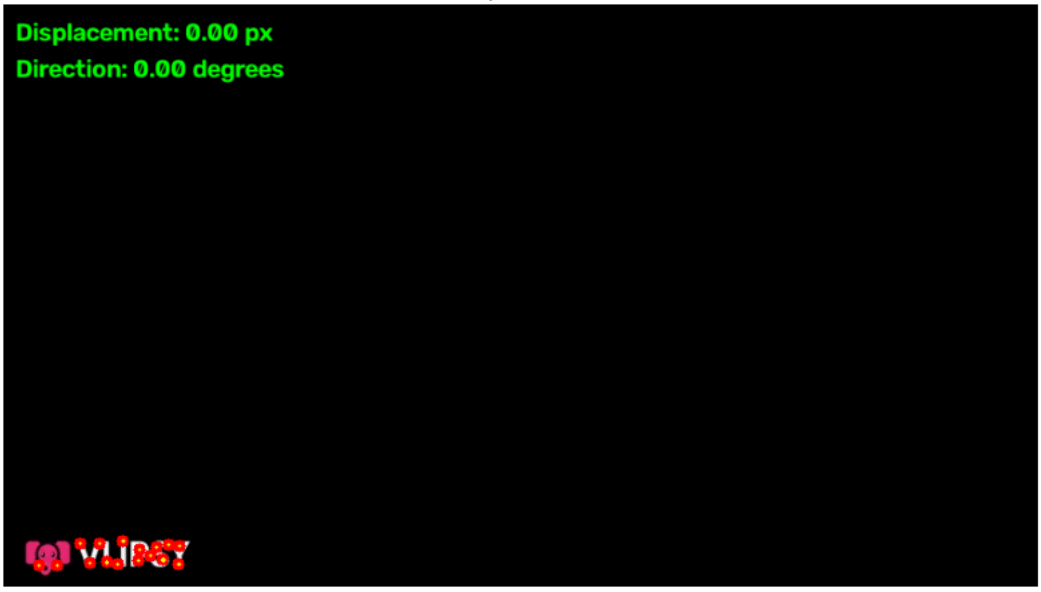
    cap.release()

print("Lucas-Kanade tracking completed.")
```

Lucas-Kanade Optical Flow - Frame 0



Lucas-Kanade Optical Flow - Frame 10



Lucas-Kanade Optical Flow - Frame 20



Lucas-Kanade Optical Flow - Frame 30





Lucas-Kanade Optical Flow - Frame 40



Lucas-Kanade Optical Flow - Frame 50

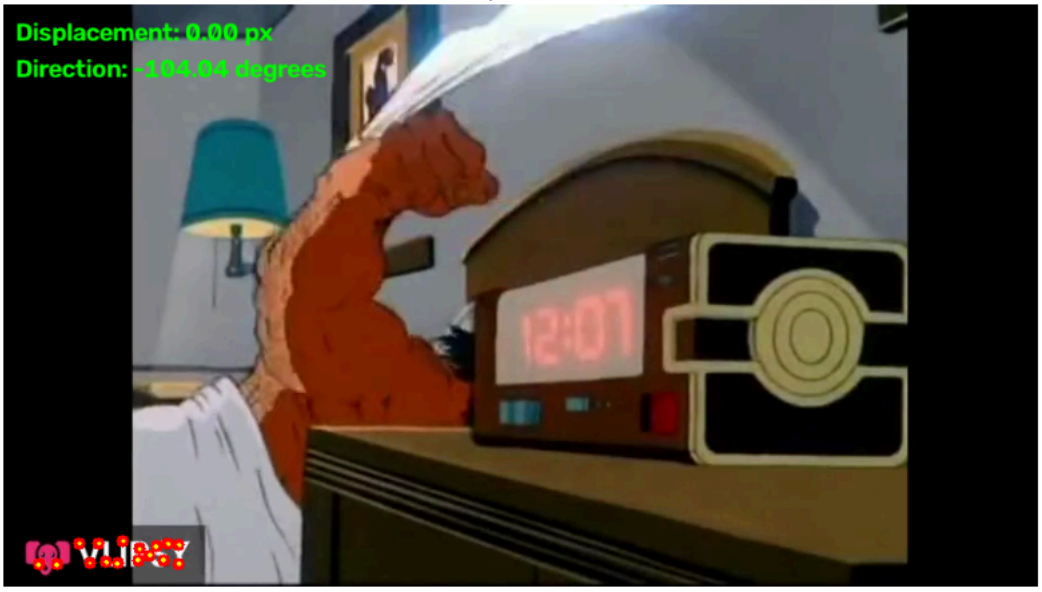


Lucas-Kanade Optical Flow - Frame 60

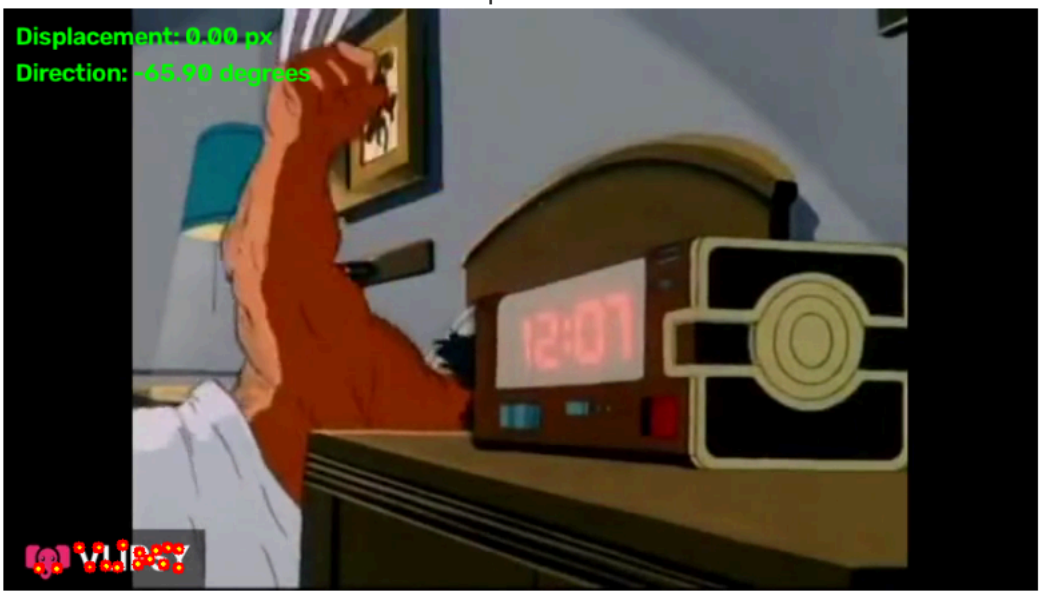




Lucas-Kanade Optical Flow - Frame 70

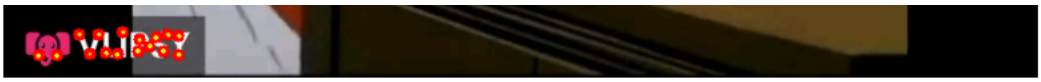


Lucas-Kanade Optical Flow - Frame 80



Lucas-Kanade Optical Flow - Frame 90





Lucas-Kanade Optical Flow - Frame 100



Lucas-Kanade Optical Flow - Frame 110



Lucas-Kanade Optical Flow - Frame 120



Lucas-Kanade Optical Flow - Frame 130



```
plt.figure(figsize=(10, 5))

plt.plot(
    trajectory_data,
    linewidth=2
)

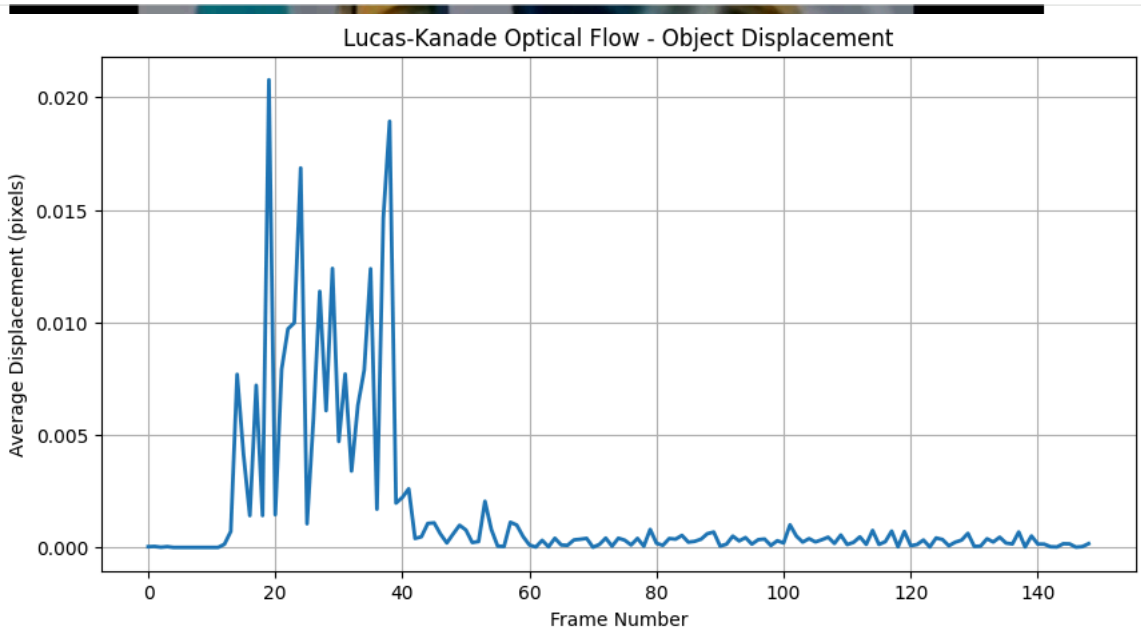
plt.title(
    "Lucas-Kanade Optical Flow - Object Displacement"
)

plt.xlabel("Frame Number")
plt.ylabel("Average Displacement (pixels)")

plt.grid(True)
plt.show()

print(
    "Average displacement:",
    round(np.mean(trajectory_data), 2),
    "pixels"
)

print(
    "Maximum displacement:",
    round(np.max(trajectory_data), 2),
    "pixels"
)
```




```
Average displacement: 0.0 pixels
Maximum displacement: 0.02 pixels
```

```
cap = cv2.VideoCapture(video_path)

ret, first_frame = cap.read()

if not ret:
    print("Unable to read video.")
else:

    previous_gray = cv2.cvtColor(
        first_frame,
        cv2.COLOR_BGR2GRAY
    )

    farneback_motion = []
    frame_number = 0

    while True:

        ret, frame = cap.read()

        if not ret:
            break

        current_gray = cv2.cvtColor(
```

```
                frame,
                cv2.COLOR_BGR2GRAY
            )

            # Farneback Dense Optical Flow
            flow = cv2.calcOpticalFlowFarneback(
                previous_gray,
                current_gray,
                None,
                0.5,
                3,
                15,
                3,
                5,
                1.2,
                0
            )

            # Magnitude and direction
            magnitude, angle = cv2.cartToPolar(
                flow[..., 0],
                flow[..., 1]
            )

            average_motion = np.mean(
                magnitude
            )

            farneback_motion.append(
                average_motion
            )

            # Create HSV visualization
            hsv = np.zeros_like(frame)

            hsv[..., 1] = 255

            hsv[..., 0] = (
                angle * 180 / np.pi / 2
            )

            hsv[..., 2] = cv2.normalize(
                magnitude,
                None,
                0,
                255,
                cv2.NORM_MINMAX
            )

            flow_visual = cv2.cvtColor(
                hsv,
                cv2.COLOR_HSV2RGB
            )

            cv2.putText(
                flow_visual,
                f"Motion: {average_motion:.2f}",
                (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (255, 255, 255),
                2
            )

            if frame_number % 10 == 0:

                plt.figure(figsize=(10, 6))
                plt.imshow(flow_visual)
                plt.title(
                    f"Farneback Dense Optical Flow - Frame {frame_number}"
                )
                plt.axis("off")
                plt.show()

            previous_gray = current_gray.copy()

            frame_number += 1

        cap.release()

    print("Farneback Optical Flow completed.")
```

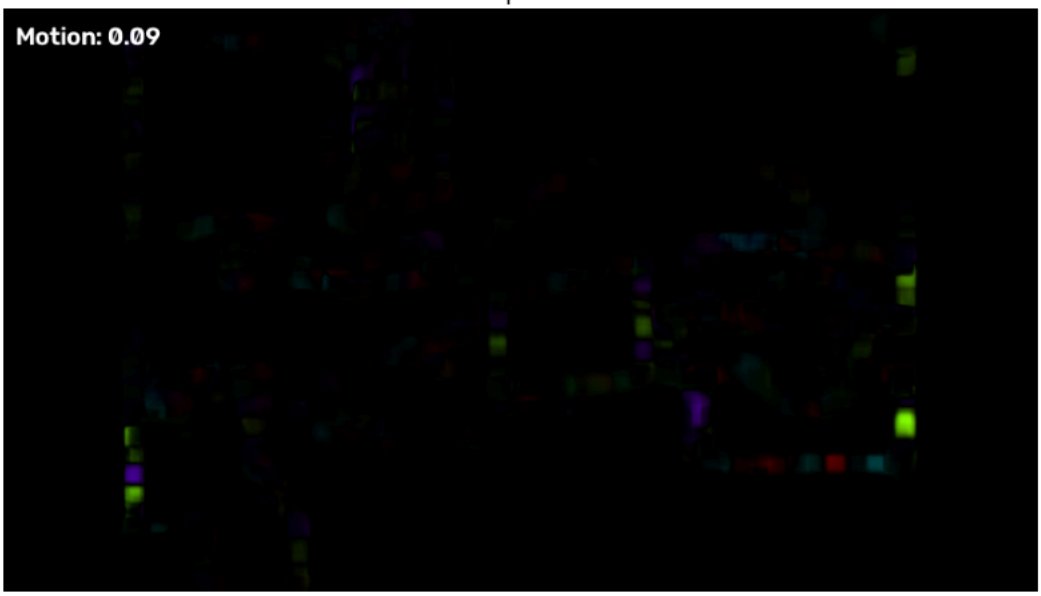
Farneback Dense Optical Flow - Frame 0



Farneback Dense Optical Flow - Frame 10



Farneback Dense Optical Flow - Frame 20



Farneback Dense Optical Flow - Frame 30





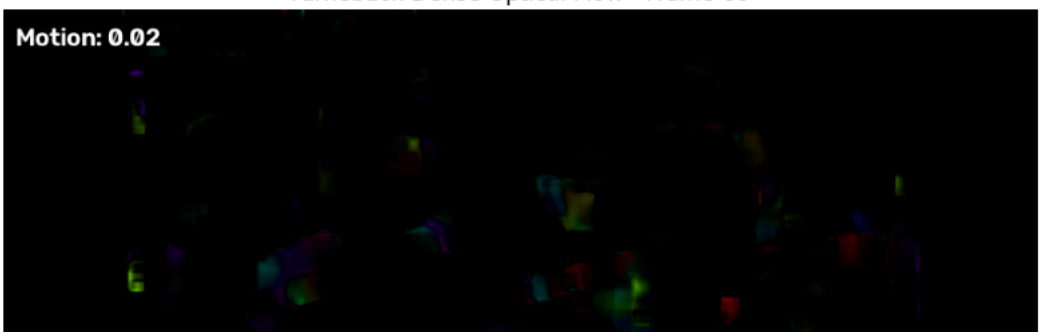
Farneback Dense Optical Flow - Frame 40

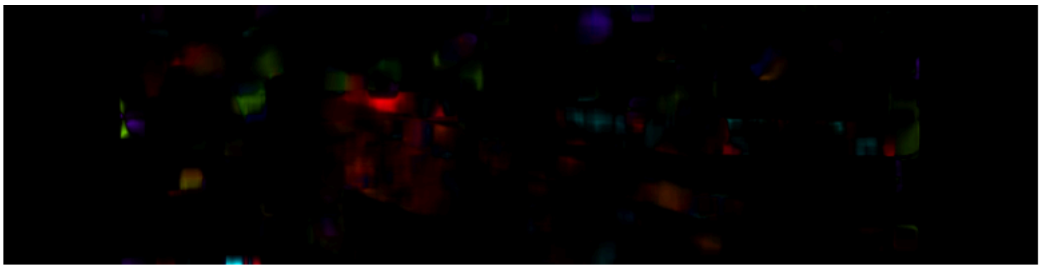


Farneback Dense Optical Flow - Frame 50



Farneback Dense Optical Flow - Frame 60

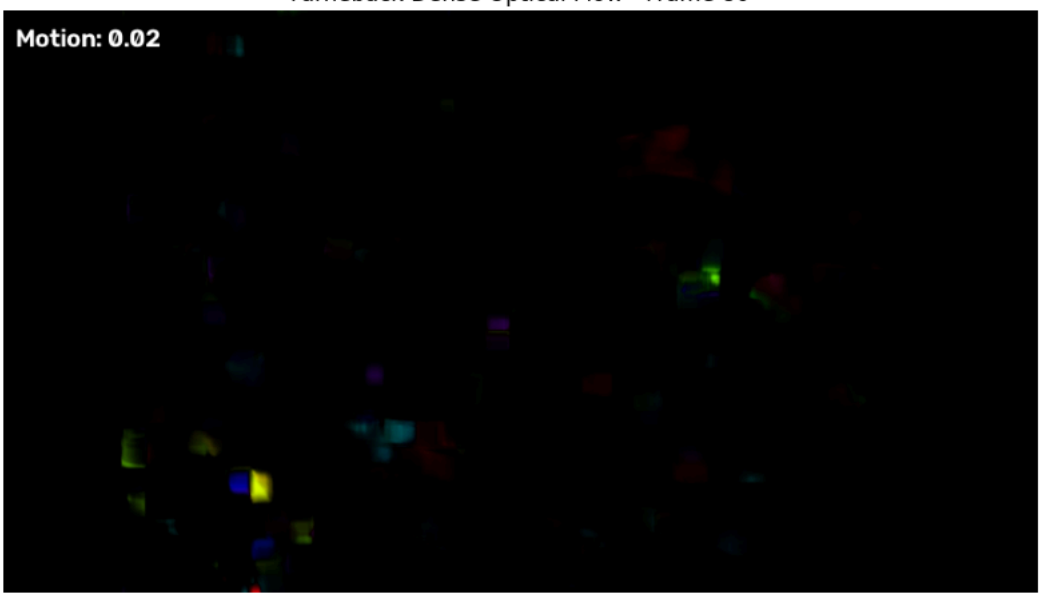




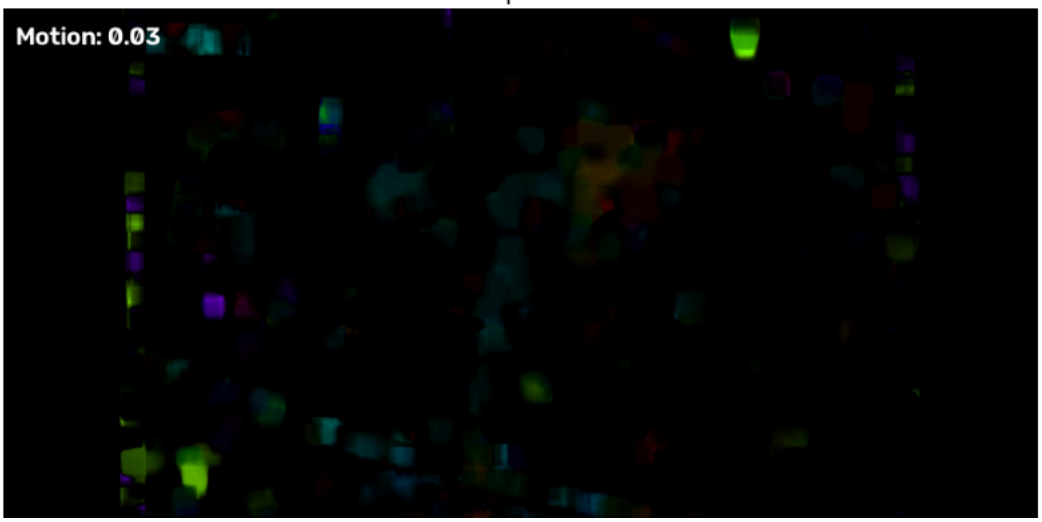
Farneback Dense Optical Flow - Frame 70

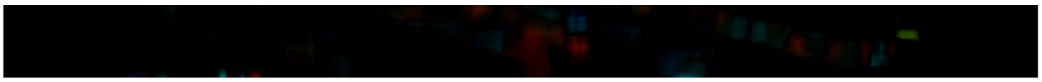


Farneback Dense Optical Flow - Frame 80

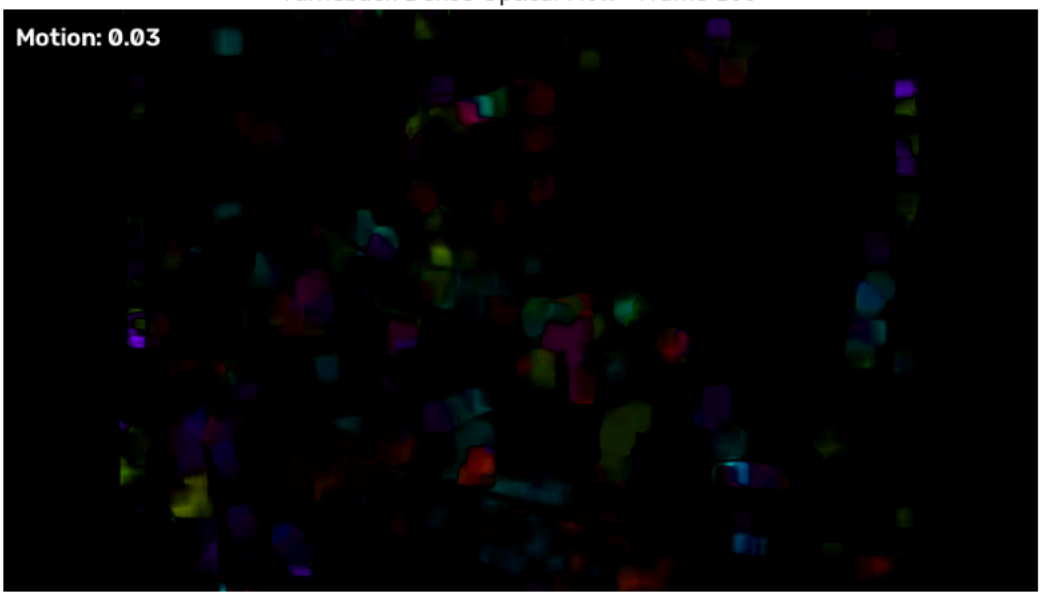


Farneback Dense Optical Flow - Frame 90

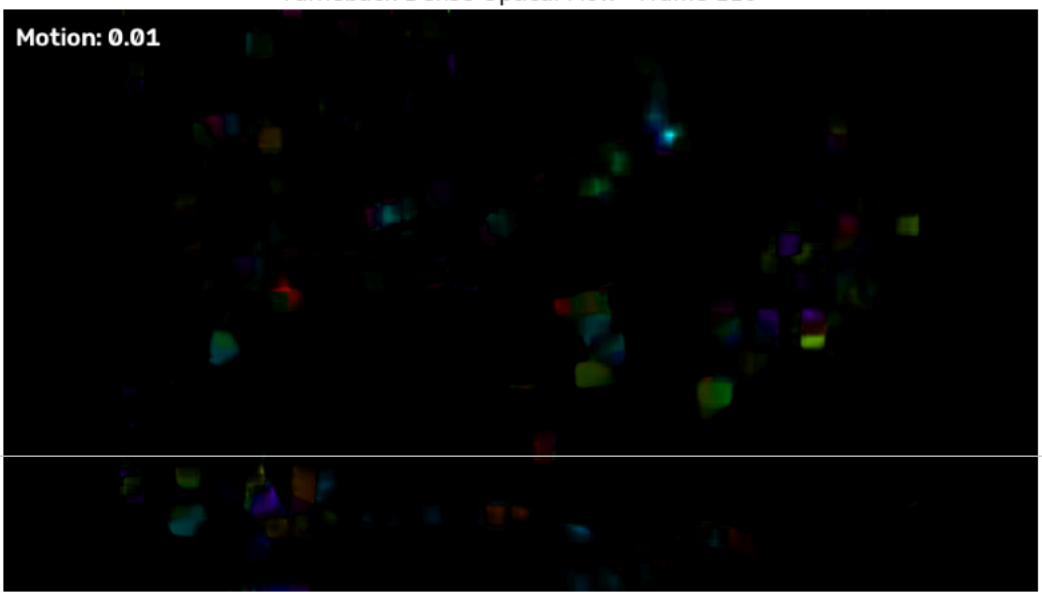




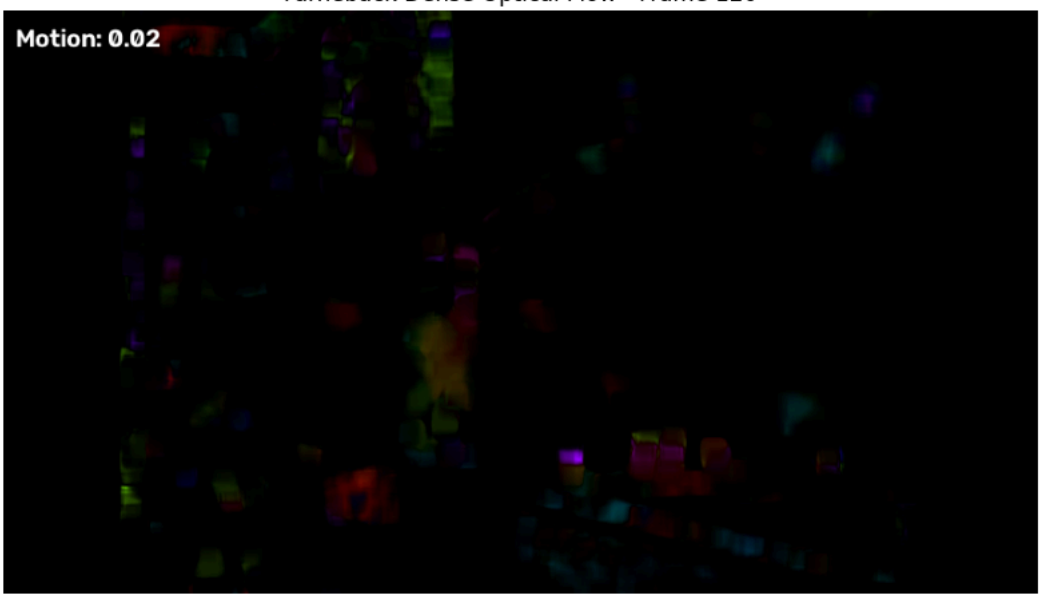
Farneback Dense Optical Flow - Frame 100



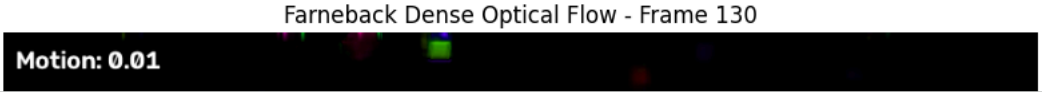
Farneback Dense Optical Flow - Frame 110



Farneback Dense Optical Flow - Frame 120



Farneback Dense Optical Flow - Frame 130



```
plt.figure(figsize=(10, 5))

plt.plot(
    farneback_motion,
    linewidth=2
)

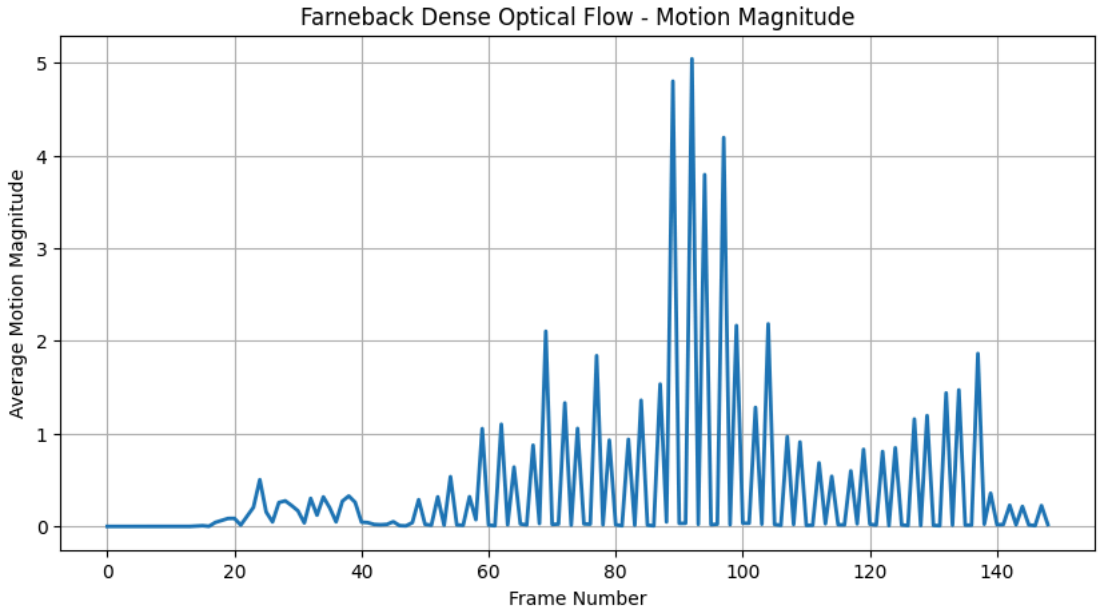
plt.title(
    "Farneback Dense Optical Flow - Motion Magnitude"
)

plt.xlabel("Frame Number")
plt.ylabel("Average Motion Magnitude")

plt.grid(True)
plt.show()

print(
    "Average motion magnitude:",
    round(np.mean(farneback_motion), 2)
)

print(
    "Maximum motion magnitude:",
    round(np.max(farneback_motion), 2)
)
```




```
Average motion magnitude: 0.4
Maximum motion magnitude: 5.04
```

```
print("OPTICAL FLOW COMPARISON")
print("=" * 45)

print("\nLucas-Kanade Optical Flow")
print("- Sparse optical flow")
print("- Tracks selected feature points")
```